

個人履歷 + 作品集網站

React + TypeScript • Vite • TailwindCSS • GitHub Actions CI/CD

部署位置 <https://brizxz.github.io/>

專案名稱	brizxz.github.io (個人作品集網站)
類型	純靜態單頁應用(SPA),內建自動化測試與部署管線
原始碼	https://github.com/brizxz/brizxz.github.io
本機位置	~/projects/github_page(WSL 原生檔案系統)
文件版本	v1.0

1. 專案概述

這是一個對外公開、隨時可分享的個人品牌網站,同時作為練習「現代軟體工程流程」的載體。網站本身是純靜態的單頁應用,但圍繞它建立了一條完整的 CI/CD 管線——程式碼品質檢查、型別檢查、自動化測試與自動部署,每次推送到 `main` 都會自動驗證並在一兩分鐘內上線。

核心目標

- 用資料驅動的方式呈現個人簡介、技能、作品集與履歷。
- 支援深色 / 淺色主題切換,並做到手機 / 平板 / 桌機的 RWD 響應式排版。
- 把工程品質「自動化」——讓 CI 當守門員,沒過檢查就不准合併。
- 部署零維運:合併即上線,不需手動處理伺服器或部署 token。

明確不做(避免範圍蔓延)

- 後端 / 資料庫:純靜態,內容以 TypeScript 設定檔管理。
- 使用者登入、CMS 後台:改內容直接改資料檔即可。

2. 技術選型

類別	選擇	理由
建置工具	Vite 5	啟動快、設定簡單,靜態輸出最適合 GitHub Pages。
框架 / 語言	React 18 + TypeScript	生態成熟;型別讓 CI 多一道有意義的關卡。
樣式	TailwindCSS 3	開發速度快,搭配自訂 keyframes 做進場動畫與漸層。
路由	React Router(HashRouter)	避開 GitHub Pages 對 SPA deep link 的 404 問題。
程式碼品質	ESLint + Prettier	靜態檢查 + 統一格式,避免無意義的 diff。
測試	Vitest + React Testing Library	與 Vite 原生整合,速度快;測元件行為而非實作細節。
CI/CD	GitHub Actions + deploy-pages	與 GitHub Pages 原生整合,官方 action 免自己處理 token。

為什麼用 HashRouter? GitHub Pages 是純靜態託管,沒有伺服器端 rewrite。直接造訪 `/projects` 會找不到檔案而回 404。HashRouter 把路由放在 `#` 之後(`/#/projects`),全交給前端處理,從根本避開問題。

3. 專案結構

```
github_page/
├── .github/workflows/
│   ├── ci.yml          # PR 檢查:format → lint → typecheck → test → build
│   └── deploy.yml       # push main 後建置並部署到 GitHub Pages
├── public/
│   ├── resume.pdf       # 可下載的 PDF 履歷
│   └── favicon.svg
├── src/
│   ├── components/      # Header, Hero, ProjectCard, ThemeToggle, Reveal
│   │   └── __tests__/   # 元件測試
│   ├── pages/           # Home, Projects, Resume
│   ├── data/            # profile.ts / projects.ts / experience.ts(內容)
│   ├── hooks/           # useTheme, useScrollReveal
│   ├── App.tsx          # HashRouter + 路由骨架
│   ├── main.tsx
│   └── index.css        # Tailwind + 自訂樣式
├── vite.config.ts       # base: "/"(部署到 username.github.io)
├── tailwind.config.js   # darkMode: "class" + 動畫 keyframes
├── vitest.config.ts     # jsdom 測試環境
└── package.json
```

資料與畫面分離

所有會變動的內容都抽到 `src/data/` : `profile.ts` (姓名、簡介、技能、社群)、`projects.ts` (作品集,含語言、stars、漸層封面色)、`experience.ts` (學歷與專案時間軸)。改內容不需要動畫面邏輯,也方便寫測試。

4. 網站功能

區塊	說明
Hero 首頁	漸層標題、狀態徽章、CTA(下載履歷 / 看作品)、社群連結,搭配光量浮動與淡入進場動畫。
關於我 / 技能	自我介紹與技能標籤(hover 變色),捲入畫面時淡入。
經歷時間軸	以學歷 / 專案分類的垂直時間軸,漸層節點。
作品集	專案卡片:漸層封面 + emoji、語言標籤、stars、技術標籤、GitHub 連結,hover 浮起。
履歷頁	結構化的網頁版履歷(簡介 / 學歷 / 專案 / 技能),並提供 PDF 下載。
主題切換	深色 / 淺色,記憶於 <code>localStorage</code> ,首次跟隨系統偏好。

無障礙與動效: 互動元素皆有 `aria-label`; 並透過 `prefers-reduced-motion` 在使用者關閉動畫偏好時停用所有動效。

5. CI/CD 管線

ci.yml — Pull Request 守門員

觸發於對 `main` 的 PR,依「快速失敗」原則由快到慢排列:

```
format:check → lint → typecheck → test → build
```

任何一步失敗就擋下合併,確保進入 `main` 的程式碼都通過驗證。

deploy.yml — 自動部署

觸發於 push 到 `main` (也支援手動 `workflow_dispatch`)。分成 build 與 deploy 兩個 job:建置出 `dist/` → 上傳成 Pages artifact → 以官方 `actions/deploy-pages` 發布。設定了 `concurrency` 避免多個部署互相覆蓋。

關卡	工具	通過標準
格式	Prettier	無格式差異
程式碼品質	ESLint	0 error
型別	tsc	0 type error
單元 / 元件	Vitest	全綠(目前 9 個測試)
建置	Vite	成功產出 dist
部署	deploy-pages	網站可正常存取

6. GitHub Pages 部署原理

base 路徑

本站部署到使用者主頁 repo `brizxz.github.io`, 網址在根目錄, 因此 `vite.config.ts` 的 `base` 設為 `"/"`。若改部署到一般 repo(`username.github.io/repo/`), 必須改成 `"/repo/"`, 否則 JS / CSS 會 404、整頁空白。

實戰踩雷:Source 設定。 部署初期網站一片空白, 原因是 GitHub Pages 的 *Source* 仍是「Deploy from a branch」, 它直接上架 repo 根目錄的原始 `index.html` (指向 `/src/main.tsx` 原始碼, 瀏覽器無法執行)。解法: 到 **Settings → Pages → Source** 改成 **GitHub Actions**, 並重新觸發一次部署, 改由 `deploy.yml` 上架建置後的成品。

7. 本機開發

```
npm install      # 安裝依賴
npm run dev      # 開發伺服器 http://localhost:5173
npm run build    # 型別檢查 + 建置到 dist/
npm run test     # 跑單元 / 元件測試
npm run lint     # ESLint
npm run format   # Prettier 自動格式化
```

WSL 環境提醒: 在 Windows 掛載點 `/mnt/c` 下安裝 `node_modules` 時, npm 解壓大型原生 binary(esbuild、rollup 等) 會被截斷損毀, 導致 build 失敗。解法是把整個專案放在 WSL 原生路徑(本專案位於 `~/projects/github_page`), 安裝與建置就完全正常。

8. 後續可擴充

- 把佔位 / 待確認的個人資訊(真實姓名、學歷年份、LinkedIn)補上。
- 放上真正的 `public/resume.pdf`。
- 加入 Playwright E2E 測試與 Lighthouse CI(效能 / 無障礙自動關卡)。
- 補 Open Graph 預覽圖, 讓社群分享有縮圖。
- 在 GitHub 設定 `main` 分支保護, 要求 CI 通過才能合併。

技術標籤總覽

React 18 TypeScript Vite TailwindCSS React Router Vitest ESLint Prettier GitHub Actions GitHub Pages